

HDOC Day-30 — 2D Array Operations in C

Problem Statement

Write a **menu-driven C program** to perform the following operations on a given 2D array:

1. Find the sum of all elements in the matrix.
 2. Find the sum of each row and store the sums in a 1D array.
 3. Find and display the transpose of the matrix.
 4. Exit.
-

1. Algorithm

Step 1: Start the program.

Step 2: Declare a 2D array `a`, variables `rows`, `cols`, `i`, `j`, `choice`, and a 1D array `rowSum`.

Step 3: Input the number of rows and columns.

Step 4: Input all elements of the matrix using nested loops.

Step 5: Display the menu:

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit

Step 6: Accept the user's choice.

Step 7: Use a `switch` statement:

- **Case 1:** Initialize `sum = 0`. Traverse the complete matrix and add every element to `sum`. Display the total sum.
- **Case 2:** For each row, initialize its sum to 0, add all elements of that row, and store the result in `rowSum[i]`. Display each row sum and the row-sum array.
- **Case 3:** Display the transpose by interchanging the row and column indices, i.e. display `a[j][i]`.
- **Case 4:** Exit the program.
- **Default:** Display "Invalid choice!".

Step 8: Repeat the menu until the user selects option 4.

Step 9: Stop.

2. Test Case Table

Test Case	Matrix	Choice	Expected Result
TC-1	1 2 3 / 4 5 6	1	Sum = 21
TC-2	1 2 3 / 4 5 6	2	Row sums = 6, 15
TC-3	1 2 3 / 4 5 6	3	Transpose = 1 4 / 2 5 / 3 6
TC-4	Any valid matrix	4	Program exits
TC-5	Any valid matrix	7	Invalid choice!
TC-6	-1 2 / 3 -4	1	Sum = 0

3. C Program

```
#include <stdio.h>

int main()
{
    int a[10][10], rowSum[10];
    int rows, cols, i, j, choice;
    int sum;

    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    printf("Enter the matrix elements:\n");

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < cols; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }

    do
    {
        printf("\n1. Sum of all elements");
        printf("\n2. Sum of each row");
        printf("\n3. Transpose of matrix");
        printf("\n4. Exit");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
```

```

sum = 0;

for (i = 0; i < rows; i++)
{
    for (j = 0; j < cols; j++)
    {
        sum = sum + a[i][j];
    }
}

printf("Sum of all elements = %d\n", sum);
break;

case 2:
printf("Row sums:\n");

for (i = 0; i < rows; i++)
{
    rowSum[i] = 0;

    for (j = 0; j < cols; j++)
    {
        rowSum[i] = rowSum[i] + a[i][j];
    }

    printf("Sum of row %d = %d\n",
        i + 1, rowSum[i]);
}

printf("The row-sum array contains: [");

for (i = 0; i < rows; i++)
{
    printf("%d", rowSum[i]);

    if (i < rows - 1)
        printf(", ");
}

printf("]\n");
break;

case 3:
printf("Transpose of the matrix:\n");

for (j = 0; j < cols; j++)
{
    for (i = 0; i < rows; i++)
    {
        printf("%d\t", a[i][j]);
    }

    printf("\n");
}
break;

case 4:

```

```

        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid choice! Please enter 1 to 4.\n");
    }

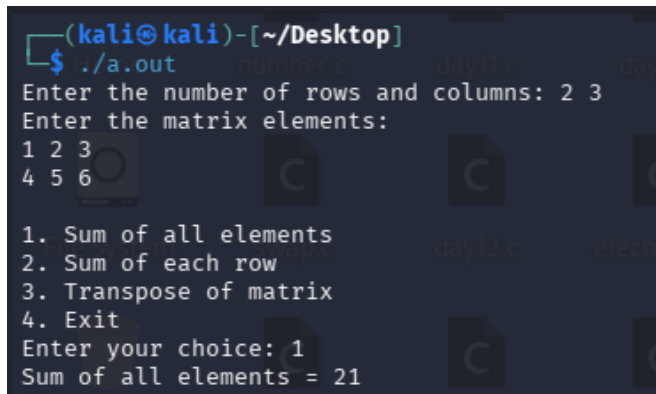
    } while (choice != 4);

    return 0;
}

```

4. Evaluation Against Prepared Test Cases

Test Case 1 — Sum of All Elements



```

(kali㉿kali)-[~/Desktop]
$ ./a.out
Enter the number of rows and columns: 2 3
Enter the matrix elements:
1 2 3
4 5 6

1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: 1
Sum of all elements = 21

```

Input matrix:

```

1 2 3
4 5 6

```

Choice: 1

Calculation:

1+2+3+4+5+6=21

Program output:

Sum of all elements = 21

Expected: 21

Actual: 21

Result: ✓ PASS

Test Case 2 — Sum of Each Row

```
1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: 2
Row sums:
Sum of row 1 = 6
Sum of row 2 = 15
The row-sum array contains: [6, 15]
```

For the same matrix:

Row 1=1+2+3=6

Row 2=4+5+6=15

Program output:

```
Row sums:
Sum of row 1 = 6
Sum of row 2 = 15
The row-sum array contains: [6, 15]
```

Expected: [6, 15]

Actual: [6, 15]

Result: ✓ PASS

Test Case 3 — Transpose

```
1. Sum of all elements
2. Sum of each row
3. Transpose of matrix
4. Exit
Enter your choice: 3
Transpose of the matrix:
1      4
2      5
3      6
```

Original matrix:

```
1 2 3
4 5 6
```

The rows become columns, therefore:

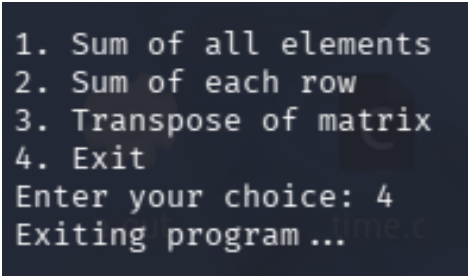
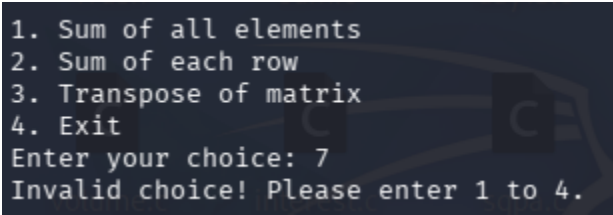
```
1    4
2    5
3    6
```

Expected: Correct 3×2 transpose

Actual: Correct 3×2 transpose

Result: ✓ PASS

Test Cases 4–6

Test Case	Expected	Actual	Status
Choice 4	Exit program	Program exits	✓ PASS
			
Choice 7	Invalid choice message	Invalid choice message displayed	✓ PASS
			
Matrix $\{-1, 2\}, \{3, -4\}$, Choice 1	Sum = 0	Sum = 0	✓ PASS

Final Result

All prepared test cases passed successfully. The program correctly performs the required 2D matrix operations using nested loops, a switch statement, and a menu-driven structure.